



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS 226 - Concurrent Systems

Practical 2 Specification

N-Thread Mutual Exclusion

Release Date: 10 August 2026

Submission Deadline: 23 August 2026

1 Overview

The purpose of this practical is to develop an understanding of mutual exclusion algorithms that support more than two concurrent threads.

In Practical 1, you implemented mutual exclusion algorithms designed for two threads. In this practical, you will implement and investigate two algorithms that generalise mutual exclusion to an arbitrary number of threads:

- Filter Lock
- Bakery Lock

You will use these algorithms to coordinate multiple threads competing for access to a shared resource.

The emphasis of this practical is on understanding how these algorithms provide mutual exclusion, how threads interact while attempting to acquire a lock, and how the two algorithms differ in their approach.

2 Learning Outcomes

By completing this practical you should be able to:

- Implement a mutual exclusion algorithm for more than two threads.
- Explain how the Filter Lock coordinates multiple competing threads.
- Explain how the Bakery Lock establishes an ordering between threads.
- Demonstrate that a shared resource is accessed by only one thread at a time.
- Explain the role of the `level`, `victim`, `flag`, and `label` variables used by the algorithms.
- Compare the Filter Lock and Bakery Lock approaches to mutual exclusion.

3 Requirements

This practical is to be completed in groups of **three**. Group members are expected to contribute equally to the implementation, and all group members should be able to explain any part of the solution during the demonstration.

No starter implementation of either lock will be provided.

Your solution must contain the following two lock implementations:

1. Filter Lock
2. Bakery Lock

Both locks must support an arbitrary number of threads specified when the lock is created.

To allow you to focus on the implementation and understanding of the algorithms, several supporting classes and files will be provided.

The following files are provided:

- **Counter** – a shared counter that will be accessed by the concurrent threads.
- **VolatileInt** – a provided wrapper for integer values that supports volatile access between threads.
- **VolatileBoolean** – a provided wrapper for boolean values that supports volatile access between threads.
- **Lock** – an interface defining the operations that a lock implementation must provide.
- **FilterLock** – a skeleton implementation of the Filter Lock. The required methods and structure will be provided, but you are responsible for completing the implementation of the algorithm.
- **BakeryLock** – a skeleton implementation of Lamport’s Bakery Lock. The required methods and structure will be provided, but you are responsible for completing the implementation of the algorithm.
- **Main** – a provided program that creates the required threads, runs the locks, and displays the results of the concurrent execution.

You are expected to use the provided classes as part of your solution. You should not modify the supporting classes unless explicitly instructed to do so.

The primary task of this practical is to complete the **Filter Lock** and **Bakery Lock** implementations and demonstrate that they provide mutual exclusion when used by multiple concurrent threads.

4 Demonstration

Students are required to demonstrate their implementation to a tutor during their scheduled practical session.

During the demonstration, students must:

- Demonstrate the Filter Lock using four concurrent threads.
- Demonstrate the Bakery Lock using four concurrent threads.
- Demonstrate that the shared counter reaches the expected value.

- Clearly identify the variables used by each algorithm and explain their purpose.

All group members must be present for the demonstration and may be asked questions individually.

Students should be prepared to explain their implementation rather than simply demonstrate that the program produces the expected output.

5 Upload Checklist

Before submitting, ensure that your group has included all of the following files:

- `FilterLock.java`
- `BakeryLock.java`
- `README.txt`

6 Submission

Your solution will be submitted via **ClickUp**. **All group members are required submit.** Your submission should include:

- Source code mentioned in the upload checklist.
- The student numbers of all group members.
- Clear comments identifying the implementation of the Filter Lock and Bakery Lock.

All required source files must be placed in a single ZIP archive. The archive must be named according to the following format: `uXXXXXXXX.zip` where `XXXXXXXX` is your student number.

No late submissions will be accepted!

7 Marking Rubric

Assessment Item	Marks
Filter Lock implementation	2
Bakery Lock implementation	2
Conceptual Question 1	3
Conceptual Question 2	3
Total	10

